

BeamLib Theory Manual

Chapter 1: Euler–Bernoulli 2D Beam Element

BeamLib Development Team

Phase 1, Batch 2A

1 Scope and Assumptions

This chapter derives the 2-node planar Euler–Bernoulli (EB) beam element used in BeamLib for slender beams ($L/h > 20$). The element operates in the global xz -plane.

Kinematic assumptions.

1. Plane sections remain plane after deformation.
2. Cross-section normals remain normal to the deformed beam axis (no shear deformation; this is the defining EB assumption).
3. Displacements and rotations are infinitesimal (linear theory).
4. Material is linear elastic, isotropic, homogeneous within the section.

The element is restricted to in-plane bending about the y -axis with axial extension along the local x -axis. Torsion and out-of-plane bending are not modeled here (they are 3D effects, handled in EB3D).

2 Sign and DOF Conventions

2.1 Coordinate system

We use a right-handed Cartesian frame with axes (x, y, z) satisfying $\hat{x} \times \hat{y} = \hat{z}$. The 2D problem lives in the xz -plane:

- x : in-plane axial reference direction.
- z : in-plane transverse direction.
- y : out-of-plane axis, oriented so that $(\hat{x}, \hat{y}, \hat{z})$ form a right-handed frame ($+\hat{y}$ points *out* of the page in the standard view with $+\hat{x}$ rightward and $+\hat{z}$ upward).

Important. Positive θ_y in BeamLib is defined by the structural-mechanics convention $\theta_y = dw/dx$ (see §2.4). This is *opposite* in sign to the strict right-hand-rule rotation about $+\hat{y}$. Throughout this manual “ θ_y ” means the BeamLib structural convention unless explicitly noted.

2.2 Per-node DOFs

At every node i we store three DOFs in the order

$$\mathbf{d}_i = \begin{bmatrix} u_{x,i} \\ u_{z,i} \\ \theta_{y,i} \end{bmatrix}.$$

Positive u_x is in the $+x$ direction (global). Positive u_z is in the $+z$ direction (global). Positive θ_y follows the BeamLib structural convention $\theta_y = dw/dx$ (see §2.4; opposite sign vs. right-hand rule about $+\hat{y}$).

2.3 Element DOF vector

A 2-node EB2D element has 6 DOFs ordered as

$$\mathbf{d}^e = \begin{bmatrix} \mathbf{d}_A \\ \mathbf{d}_B \end{bmatrix} = [u_{xA}, \quad u_{zA}, \quad \theta_{yA}, \quad u_{xB}, \quad u_{zB}, \quad \theta_{yB}]^\top.$$

Indices in this chapter run $0, \dots, 5$ to match the C++ implementation.

2.4 Sign convention table

Quantity	Symbol	Positive direction
Axial displacement	u_x	along $+\hat{x}$ (global x)
Transverse displacement	u_z	along $+\hat{z}$ (global z)
Section rotation	θ_y	structural convention: $\theta_y = dw/dx$ (positive slope $\Rightarrow$ positive)
Axial force on a node	F_x	along $+\hat{x}$
Transverse force on a node	F_z	along $+\hat{z}$
Nodal moment	M_y	work-conjugate to θ_y (same convention; positive M_y produces
Slope-rotation relation	$\theta_y(x) = dw/dx$	with $w \equiv u_z$

Table 1: Sign conventions for the EB2D element.

Justification of $\theta_y = dw/dx$ (structural convention). BeamLib adopts the standard structural-mechanics convention: a positive section rotation θ_y corresponds to a positive slope dw/dx . Geometrically, in the xz -plane as drawn ($+\hat{x}$ rightward, $+\hat{z}$ upward), this is a counter-clockwise rotation of the cross-section as the viewer reads the figure.

This is the *opposite* sign from a strict right-hand-rule rotation about $+\hat{y}$. By Rodrigues' formula,

$$R_y^{\text{RH}}(\theta)\hat{x} = (\cos \theta, 0, -\sin \theta),$$

so a positive right-hand-rule rotation tilts the cross-section normal toward $-\hat{z}$, not $+\hat{z}$. The BeamLib convention is therefore equivalent to a right-hand-rule rotation about $-\hat{y}$.

We adopt the structural convention because (i) it makes bending kinematics intuitive: positive slope $\Rightarrow$ positive rotation, no mental flip needed when reading a diagram; (ii) it matches the standard Hermite shape function $H_2(\xi) = L(\xi - 2\xi^2 + \xi^3)$, whose derivative at $\xi = 0$ is $+1$, enforcing $\theta_{yA} = +du_z/dx$ at node A ; (iii) it matches the MATLAB `JaphyBeam` reference and most structural-mechanics textbooks. The convention propagates consistently to every k_l sign in §5, the residual definition $\mathbf{r}_l = \mathbf{k}_l \mathbf{d}_l$, and the verification signs in §10.

Cross-check. For a cantilever with downward tip load $F_z < 0$: the beam droops ($u_z(L) < 0$), the slope is negative at the tip ($dw/dx < 0$), so under the BeamLib convention $\theta_y(L) = dw/dx < 0$. The test `test_eb2d_patch.cpp` asserts this sign explicitly.

3 Strain–Displacement Relations

Let $u(x, z)$, $w(x, z)$ denote total displacement of a material point. EB kinematics:

$$u(x, z) = u_x(x) - z \theta_y(x) = u_x(x) - z \frac{dw}{dx}, \quad w(x, z) = u_z(x).$$

The axial strain is

$$\varepsilon_{xx} = \frac{\partial u}{\partial x} = \frac{du_x}{dx} - z \frac{d^2 w}{dx^2}.$$

The bending curvature is $\kappa = d^2 w / dx^2 = d^2 u_z / dx^2$.

The strain energy density per unit length, after integrating over the cross-section, is

$$\frac{1}{2} \int_A E \varepsilon_{xx}^2 dA = \frac{1}{2} EA \left(\frac{du_x}{dx} \right)^2 + \frac{1}{2} EI_z \left(\frac{d^2 u_z}{dx^2} \right)^2,$$

where $I_z = \int_A z^2 dA$ is the second moment of area about the y -axis (the axis perpendicular to the bending plane, i.e. $\hat{y}$). **In BeamLib we store this as `SectionProperties::Iz`**, consistent with the 2D xz -plane bending convention.

Section property used. 2D EB bending in the xz -plane uses `props.Iz`. The 2D–3D cross-validation table in PROJECT_SPEC §13.6 maps 2D `Iz` $\leftrightarrow$ 3D `Iy` when comparing the same physical scenario.

4 Shape Functions

Use a 2-node element on the natural coordinate $\xi = x/L \in [0, 1]$.

4.1 Axial (linear)

$$N_1^a(\xi) = 1 - \xi, \quad N_2^a(\xi) = \xi.$$

Axial displacement:

$$u_x(\xi) = N_1^a(\xi) u_{xA} + N_2^a(\xi) u_{xB}.$$

4.2 Transverse (Hermite cubic)

Four cubic Hermite functions interpolate u_z and $\theta_y = du_z/dx$ at the two ends:

$$\begin{aligned} H_1(\xi) &= 1 - 3\xi^2 + 2\xi^3, & H_2(\xi) &= L(\xi - 2\xi^2 + \xi^3), \\ H_3(\xi) &= 3\xi^2 - 2\xi^3, & H_4(\xi) &= L(-\xi^2 + \xi^3). \end{aligned}$$

Transverse displacement:

$$u_z(\xi) = H_1 u_{zA} + H_2 \theta_{yA} + H_3 u_{zB} + H_4 \theta_{yB}.$$

Verification at the endpoints (using $\theta_y = du_z/dx = (1/L) du_z/d\xi$):

$$u_z(0) = u_{zA}, \quad u_z(1) = u_{zB}, \quad \left. \frac{du_z}{dx} \right|_{\xi=0} = \theta_{yA}, \quad \left. \frac{du_z}{dx} \right|_{\xi=1} = \theta_{yB}.$$

4.3 Curvature operator

Let $\mathbf{d}_b = (u_{zA}, \theta_{yA}, u_{zB}, \theta_{yB})^\top$. Then

$$\kappa(\xi) = \frac{d^2 u_z}{dx^2} = \mathbf{B}_b(\xi) \mathbf{d}_b,$$

with

$$\mathbf{B}_b(\xi) = \frac{1}{L^2} \begin{bmatrix} -6 + 12\xi & L(-4 + 6\xi) & 6 - 12\xi & L(-2 + 6\xi) \end{bmatrix}.$$

5 Element Stiffness Matrix Derivation

5.1 Axial stiffness

$$\mathbf{k}_{2 \times 2}^a = \int_0^L EA \begin{bmatrix} N_1'^2 & N_1' N_2' \\ N_1' N_2' & N_2'^2 \end{bmatrix} dx = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

5.2 Bending stiffness

$$\mathbf{k}_{4 \times 4}^b = \int_0^L EI_z \mathbf{B}_b^\top \mathbf{B}_b dx.$$

Performing the integral yields the well-known result

$$\mathbf{k}^b = \frac{EI_z}{L^3} \begin{bmatrix} 12 & 6L & -12 & 6L \\ 6L & 4L^2 & -6L & 2L^2 \\ -12 & -6L & 12 & -6L \\ 6L & 2L^2 & -6L & 4L^2 \end{bmatrix}.$$

5.3 Assembled element stiffness $\mathbf{k}_l$ in DOF order $[u_{xA}, u_{zA}, \theta_{yA}, u_{xB}, u_{zB}, \theta_{yB}]$

Mapping the axial block to indices (0, 3) and the bending block to indices (1, 2, 4, 5), the nonzero entries of the local 6×6 stiffness $\mathbf{k}_l$ are:

$$\begin{aligned} k_l(0, 0) &= k_l(3, 3) = \frac{EA}{L}, & k_l(0, 3) &= k_l(3, 0) = -\frac{EA}{L}, \\ k_l(1, 1) &= k_l(4, 4) = \frac{12EI_z}{L^3}, & k_l(1, 4) &= k_l(4, 1) = -\frac{12EI_z}{L^3}, \\ k_l(1, 2) &= k_l(2, 1) = \frac{6EI_z}{L^2}, & k_l(1, 5) &= k_l(5, 1) = \frac{6EI_z}{L^2}, \\ k_l(2, 2) &= k_l(5, 5) = \frac{4EI_z}{L}, & k_l(2, 5) &= k_l(5, 2) = \frac{2EI_z}{L}, \\ k_l(2, 4) &= k_l(4, 2) = -\frac{6EI_z}{L^2}, & k_l(4, 5) &= k_l(5, 4) = -\frac{6EI_z}{L^2}. \end{aligned}$$

All other entries are zero. The matrix is symmetric.

5.4 Residual convention

For this linear element the residual (internal force vector) is

$$\boxed{\mathbf{r}_l^e = \mathbf{k}_l \mathbf{d}_l^e.}$$

The Newton residual at the global level is $\mathbf{R} = \mathbf{R}_{\text{int}} - \mathbf{F}_{\text{ext}}$ and the correction equation is $\mathbf{K} \delta = -\mathbf{R}$. With this sign, the convergent state satisfies $\mathbf{R}_{\text{int}} = \mathbf{F}_{\text{ext}}$.

6 Consistent Mass Matrix

The consistent mass matrix for EB2D uses the same shape functions:

$$\mathbf{m}_{2 \times 2}^a = \int_0^L \rho A \begin{bmatrix} N_1^a N_1^a & N_1^a N_2^a \\ N_1^a N_2^a & N_2^a N_2^a \end{bmatrix} dx = \frac{\rho AL}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix},$$

$$\mathbf{m}_{4 \times 4}^b = \int_0^L \rho A \mathbf{H}^\top \mathbf{H} dx = \frac{\rho AL}{420} \begin{bmatrix} 156 & 22L & 54 & -13L \\ 22L & 4L^2 & 13L & -3L^2 \\ 54 & 13L & 156 & -22L \\ -13L & -3L^2 & -22L & 4L^2 \end{bmatrix}.$$

EB2D mass does *not* include rotary inertia (this is the standard textbook consistent mass for EB and matches PROJECT_SPEC §3.4). Embedded in the 6×6 element DOF order:

$$\begin{aligned} m_l(0,0) &= m_l(3,3) = \frac{\rho AL}{3}, & m_l(0,3) &= m_l(3,0) = \frac{\rho AL}{6}, \\ m_l(1,1) &= m_l(4,4) = \frac{156 \rho AL}{420}, & m_l(1,4) &= m_l(4,1) = \frac{54 \rho AL}{420}, \\ m_l(1,2) &= m_l(2,1) = \frac{22L \rho AL}{420}, & m_l(1,5) &= m_l(5,1) = -\frac{13L \rho AL}{420}, \\ m_l(2,2) &= m_l(5,5) = \frac{4L^2 \rho AL}{420}, & m_l(2,5) &= m_l(5,2) = -\frac{3L^2 \rho AL}{420}, \\ m_l(2,4) &= m_l(4,2) = \frac{13L \rho AL}{420}, & m_l(4,5) &= m_l(5,4) = -\frac{22L \rho AL}{420}. \end{aligned}$$

This matrix is implemented in Batch 2A as part of `computeMass`, but the assembly path `BeamModel::assembleMass` is left for Batch 2B.

7 Coordinate Transformation

Each element is integrated in a canonical *local* frame where node *A* sits at the origin and node *B* at $(L, 0, 0)$. The transformation $\mathbf{T}$ relates global element DOFs to local DOFs.

7.1 Per-node block

Let $\Delta \mathbf{x} = \mathbf{x}_B - \mathbf{x}_A$ in global coordinates with components Δx_x and Δx_z (the y component is zero in the 2D problem). Define

$$L = \|\Delta \mathbf{x}\|, \quad c = \frac{\Delta x_x}{L}, \quad s = \frac{\Delta x_z}{L}.$$

The 3×3 per-node block is

$$\mathbf{T}_n = \begin{bmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The element-level 6×6 matrix is block-diagonal: $\mathbf{T} = \text{diag}(\mathbf{T}_n, \mathbf{T}_n)$.

7.2 Application convention

By construction $\mathbf{T}$ is orthogonal ($\mathbf{T}^\top \mathbf{T} = \mathbf{I}$). BeamLib applies the transformation as

$$\mathbf{d}_l = \mathbf{T} \mathbf{d}_g, \quad \mathbf{r}_g = \mathbf{T}^\top \mathbf{r}_l, \quad \mathbf{k}_g = \mathbf{T}^\top \mathbf{k}_l \mathbf{T}.$$

This convention matches the MATLAB reference (`JaphyBeam/JaphyEulerBernoulliBeam2D/calcTransform`) and the user spec verbatim.

Sanity checks.

- Horizontal beam ($\mathbf{x}_B - \mathbf{x}_A = (L, 0, 0)$): $c = 1, s = 0, \mathbf{T}_n = \mathbf{I}_3$, so local and global frames coincide.
- Orthogonality: $\mathbf{T}_n^\top \mathbf{T}_n = \text{diag}(c^2 + s^2, c^2 + s^2, 1) = \mathbf{I}_3$ since $c^2 + s^2 = 1$.
- Energy invariance: $\mathbf{d}_g^\top \mathbf{k}_g \mathbf{d}_g = (\mathbf{T} \mathbf{d}_g)^\top \mathbf{k}_l (\mathbf{T} \mathbf{d}_g) = \mathbf{d}_l^\top \mathbf{k}_l \mathbf{d}_l$.
- The θ_y DOF is invariant ($T_n(2, 2) = 1$) because the rotation axis $\hat{y}$ is preserved by 2D in-plane rotation about $\hat{y}$.

Note on Batch 2A scope. All Batch 2A tests use beams aligned with the global x -axis, so $\mathbf{T} = \mathbf{I}$ and the transformation is the identity. The non-trivial rotated-beam verification is deferred to Batch 2B (PROJECT_SPEC §2B.5).

7.3 Reference vector

`computeTransformation(xA, xB, refVector)` takes a `refVector` argument for API uniformity with 3D elements but **ignores it** in 2D. The 2D rotation about $+\hat{y}$ is fully determined by the in-plane direction $(\Delta x_x, \Delta x_z)$.

8 Global Assembly

For each element with connectivity (A, B) :

1. Build local $\mathbf{T}$ from $\mathbf{x}_A, \mathbf{x}_B$.
2. Gather global element DOFs $\mathbf{d}_g^e$ from nodal storage.
3. $\mathbf{d}_l^e = \mathbf{T} \mathbf{d}_g^e$.
4. Pass $\mathbf{x}_A^{loc} = (0, 0, 0), \mathbf{x}_B^{loc} = (L, 0, 0), \mathbf{d}_l^e$ to `EulerBernoulli2D::computeElement` to obtain $\mathbf{r}_l^e$ and $\mathbf{k}_l^e$.
5. $\mathbf{r}_g^e = \mathbf{T}^\top \mathbf{r}_l^e, \mathbf{k}_g^e = \mathbf{T}^\top \mathbf{k}_l^e \mathbf{T}$.
6. Scatter $\mathbf{r}_g^e, \mathbf{k}_g^e$ into the free-DOF system using the DofMap. Entries with any fixed DOF on either side are dropped (Batch 2A only supports homogeneous Dirichlet constraints; PROJECT_SPEC §11).

9 Formula-to-C++ Index Mapping

Formula symbol	C++ index	Meaning
u_{xA}	<code>dispVec[0]</code>	node-A axial displacement (global x in canonical local frame)
u_{zA}	<code>dispVec[1]</code>	node-A transverse displacement
θ_{yA}	<code>dispVec[2]</code>	node-A section rotation
u_{xB}	<code>dispVec[3]</code>	node-B axial displacement
u_{zB}	<code>dispVec[4]</code>	node-B transverse displacement
θ_{yB}	<code>dispVec[5]</code>	node-B section rotation
$k_l(0, 0)$	<code>ke(0, 0)</code>	EA/L
$k_l(3, 3)$	<code>ke(3, 3)</code>	EA/L

Formula symbol	C++ index	Meaning
$k_l(0, 3)$	<code>ke(0, 3)</code>	$-EA/L$
$k_l(1, 1)$	<code>ke(1, 1)</code>	$12EI_z/L^3$
$k_l(1, 2)$	<code>ke(1, 2)</code>	$6EI_z/L^2$
$k_l(1, 4)$	<code>ke(1, 4)</code>	$-12EI_z/L^3$
$k_l(1, 5)$	<code>ke(1, 5)</code>	$6EI_z/L^2$
$k_l(2, 2)$	<code>ke(2, 2)</code>	$4EI_z/L$
$k_l(2, 4)$	<code>ke(2, 4)</code>	$-6EI_z/L^2$
$k_l(2, 5)$	<code>ke(2, 5)</code>	$2EI_z/L$
$k_l(4, 4)$	<code>ke(4, 4)</code>	$12EI_z/L^3$
$k_l(4, 5)$	<code>ke(4, 5)</code>	$-6EI_z/L^2$
$k_l(5, 5)$	<code>ke(5, 5)</code>	$4EI_z/L$
c	<code>c</code>	$\Delta x_x/L$
s	<code>s</code>	$\Delta x_z/L$ (note: z , not y , in the xz -plane)
$T_n(0, 1)$	<code>T(0, 1)</code>	$-s$
$T_n(1, 0)$	<code>T(1, 0)</code>	$+s$

Table 2: Formula symbol to C++ index mapping for EB2D.

10 Verification: Analytical Solutions

10.1 Cantilever tip transverse load

A cantilever of length L fixed at node 0 with a transverse load F_z at the tip satisfies

$$u_z(L) = \frac{F_z L^3}{3EI_z}, \quad \theta_y(L) = \frac{F_z L^2}{2EI_z}.$$

For $F_z < 0$ (downward), both are negative (tip droops, slope is negative). With $F_z = -F$ ($F > 0$ being the magnitude),

$$u_z(L) = -\frac{FL^3}{3EI_z}, \quad \theta_y(L) = -\frac{FL^2}{2EI_z}.$$

Reference numerical values used in `test_eb2d_cantilever.cpp`: $E = 200 \times 10^9$, $A = 0.01$, $I_z = 8.333 \times 10^{-6}$, $L = 1.0$, $F = 1000$, giving $u_z(L) = -2 \times 10^{-4}$ and $\theta_y(L) = -3 \times 10^{-4}$.

10.2 Cantilever tip end-moment

A cantilever with an applied end moment M_y at the tip yields a constant curvature $\kappa = M_y/(EI_z)$ throughout the beam and

$$u_z(L) = \frac{M_y L^2}{2EI_z}, \quad \theta_y(L) = \frac{M_y L}{EI_z}.$$

The bending moment is constant along the span (no shear, pure bending).

10.3 Patch test (constant axial strain)

Apply $u_{xB} = u_0$ at the right end of an arbitrary mesh of EB2D elements aligned with $+\hat{x}$, with the left end pinned axially. Every element should report axial strain u_0/L_{total} and zero bending response.

10.4 Rigid translation

Setting $u_{xA} = u_{xB}$, $u_{zA} = u_{zB}$, $\theta_{yA} = \theta_{yB} = 0$ should give $\mathbf{r}_l = \mathbf{k}_l \mathbf{d}_l$ where the axial block contributes zero (equal axial DOFs) and bending block also gives zero (constant deflection, zero curvature, zero rotation difference). Hence $\mathbf{r}_l = \mathbf{0}$.

11 Verification Tolerance Table

Check	Tolerance
Single-element matrix entries (EA/L , $12EI/L^3$, ...)	relative 10^{-12}
Patch test rigid-translation residual	absolute 10^{-12}
Patch test constant-strain residual	absolute 10^{-10}
Cantilever tip u_z , θ_y (10 elements)	relative 10^{-9}
Pure end-moment u_z , θ_y (10 elements)	relative 10^{-9}
Constant-moment uniformity along span	relative 10^{-9}
Newton iteration count (linear case)	exactly 1 correction step
Final NR residual (linear case)	$\leq 10^{-7}$ absolute (set by NRConfig.tol)

Table 3: Verification tolerance for EB2D tests.

12 Implementation Pitfalls (for the Reviewer)

1. **Section property choice.** EB2D bending in the xz -plane uses `props.Iz`. Using `props.Iy` is a silent bug in 2D (the field exists but has no role here).
2. **s comes from Δx_z , not Δx_y .** The 2D problem lives in the xz -plane; y is out-of-plane. Reusing a generic “ xy ” rotation formula will give wrong transformations for non-horizontal beams.
3. **Residual sign.** The element returns $\mathbf{r}_l = \mathbf{k}_l \mathbf{d}_l$, not $-\mathbf{k}_l \mathbf{d}_l$. The Newton solver assembles $\mathbf{R} = \mathbf{R}_{\text{int}} - \mathbf{F}_{\text{ext}}$ and solves $\mathbf{K} \delta = -\mathbf{R}$.
4. **θ_y sign under F_z .** A downward tip load yields a negative θ_y at the tip (test `test_eb2d_patch.cpp` explicitly checks this).
5. **Bending couplings.** The signs in k_l entries connecting u_z and θ_y at the same node are positive (e.g. $k_l(1, 2) = +6EI/L^2$), while couplings between u_z at one node and θ_y at the other can be positive or negative depending on which side (see the index map above).